Guía de implementación

Sistema de registro de piezas con login por roles (trabajador / ejecutivo), hospedado en GitHub Pages con Firebase como backend.

---

1. Qué hace este sistema

Es un sitio web con tres pantallas: una pantalla de inicio de sesión, un formulario para que el **trabajador** registre el movimiento de una pieza (nombre, folio, fecha/hora, comentarios, zona de origen y zona destino), y un **dashboard** para que el **ejecutivo** consulte y filtre todos los registros. Los datos se guardan en Firestore (la base de datos de Firebase) y el sitio se publica gratis en GitHub Pages.

> Zonas de origen: Externo + Zona A a Zona Z. Zonas destino: Zona A a Zona Z (sin Externo), ya que toda pieza termina en una zona interna.

2. Crear el proyecto en Firebase y registrar la app web

1. Entra a **console.firebase.google.com** con tu cuenta de Google.
2. Clic en **"Agregar proyecto"**, ponle un nombre (ej. RegistroPiezas) y crea el proyecto.
3. Al terminar, Firebase te deja en la pantalla de bienvenida del proyecto ("Hola [tu nombre]", con tarjetas sugeridas de Gemini). Ignora esas tarjetas y busca el botón **"+ Agregar app"**, arriba junto al nombre del proyecto.
4. Firebase muestra varios íconos circulares de plataforma en este orden: iOS, Android, Web, Unity y Flutter. **No elijas Android** (aunque sea el que aparece resaltado al pasar el mouse) ni iOS. El ícono correcto es el tercero, el que tiene el símbolo **</>** (código) — ese es "Web".
5. Dale un apodo a la app (ej. "RegistroPiezas Web"). **No** marques la casilla de Firebase Hosting, ya que el sitio se va a publicar en GitHub Pages.
6. Clic en **"Registrar app"**.
7. Firebase te lleva a la pantalla **"Agrega el SDK de Firebase"**, con dos opciones: "Usar npm" o "Usar una etiqueta <script>". Elige **"Usar una etiqueta <script>"**: este proyecto no usa npm ni un empaquetador (Webpack/Rollup), carga Firebase directo desde internet (CDN), así que esa es la opción que corresponde. No necesitas copiar ningún código de ahí, el proyecto ya lo trae listo.
8. Lo único que necesitas de esa pantalla es el bloque **const firebaseConfig = { ... }** que aparece ahí (apiKey, authDomain, projectId, storageBucket, messagingSenderId, appId). Cópialo o déjalo abierto en esa pantalla — lo vas a necesitar en el paso 5.
9. Clic en "Ir a la consola" para cerrar el asistente. Ya estás de regreso en la pantalla principal del proyecto.
10. En el menú lateral entra a **Seguridad** y, dentro de ese submenú, clic en **Authentication**. Da clic en "Comenzar".
11. Dentro de Authentication, pestaña "Sign-in method", activa el proveedor **Correo electrónico/contraseña**.
12. En el menú lateral entra a **Bases de datos y almacenamiento**. Se abre un submenú con tres grupos: PostgreSQL (SQL Connect), NoSQL (Firestore, Realtime Database) y Almacenamiento de objetos

(Storage). Dentro del grupo **NoSQL**, clic en **Firestore** (no Realtime Database, no SQL Connect). Da clic en "Crear base de datos". Elige modo de producción y la región más cercana (ej. us-central).

Si más adelante necesitas volver a ver el firebaseConfig, está en el ícono de engrane (arriba del menú lateral) → **Configuración del proyecto**, sección "Tus apps".

3. Crear los usuarios (trabajador y ejecutivo)

Por ahora los usuarios se crean a mano desde la consola, sin pantalla de administración:

1. En **Authentication** → **Users**, clic en "Agregar usuario". Captura un correo y una contraseña para cada persona (ej. juan@empresa.com para un trabajador, ana@empresa.com para un ejecutivo).
2. Copia el **UID** que Firebase asigna a cada usuario (aparece en la lista de usuarios).
3. Ve a **Firestore** → **Datos** y crea una colección llamada **usuarios**.
4. Firebase abre el asistente "Inicia una colección" y te muestra un **ID de documento** generado automáticamente (algo como "61b7lcJw8HjIKAaeUYUg"). **Bórralo y pega ahí el UID** que copiaste en el paso anterior — tiene que quedar idéntico, letra por letra, porque así es como el sistema relaciona la cuenta de login con su rol.
5. Agrega dos campos, ambos de tipo **string** (así viene seleccionado por defecto):
   • **nombre** = el nombre de la persona (ej. "Juan Pérez")
   • **rol** = exactamente **trabajador** o **ejecutivo** (en minúsculas, tal como está en el código). Para agregar el segundo campo usa el botón **"Agregar campo"** antes de guardar.
6. Clic en **Guardar**.
7. Repite esto por cada persona que necesite acceso al sistema (un documento nuevo en "usuarios" por cada UID).

El sistema decide a qué pantalla mandar a cada persona leyendo este campo "rol" después de iniciar sesión.

4. Pegar las reglas de seguridad de Firestore

Estas reglas evitan que cualquier persona sin permiso lea o modifique los registros desde fuera del sitio.

1. En Firestore, ve a la pestaña **Reglas**.
2. Borra el contenido y pega el archivo **firestore.rules** que viene en la carpeta del proyecto.
3. Clic en **Publicar**.

En resumen, esas reglas permiten: cualquier usuario con sesión puede crear registros; solo el rol "ejecutivo" puede leer/listar todos los registros; nadie puede editar ni borrar registros ya creados; cada usuario solo puede leer su propio perfil.

5. Pegar la configuración de Firebase en el código

1. Abre el archivo **js/firebase-config.js** del proyecto.
2. Reemplaza los valores de ejemplo (apiKey, authDomain, projectId, storageBucket, messagingSenderId, appId) con los que copiaste del **firebaseConfig** en el paso 2.

Ejemplo de cómo se ve ese archivo ya configurado:

```
export const firebaseConfig = {
  apiKey: "AIza...",
  authDomain: "registropiezas-6039d.firebaseapp.com",
  projectId: "registropiezas-6039d",
  storageBucket: "registropiezas-6039d.firebasestorage.app",
  messagingSenderId: "151928281711",
  appId: "1:151928281711:web:abcabc"
};
```

Nota: el nombre del storageBucket cambió de formato recientemente — antes terminaba en ".appspot.com", ahora Firebase lo genera como ".firebasestorage.app". Usa el valor exacto que te muestre tu propia consola.

6. Dónde vive ese <script type="module"> dentro del proyecto

La pantalla de Firebase te mostró un bloque completo de <script type="module"> con el import de Firebase, tu firebaseConfig y el initializeApp, todo junto, pensado para alguien que empieza un proyecto desde cero. Este proyecto ya tiene esa misma idea, pero repartida en varios archivos para que sea más fácil de mantener. **No hace falta copiar ese bloque**, solo los valores del firebaseConfig (ya hecho en el paso 5).

Así es como encaja cada pieza:

- **js/firebase-init.js** es el único lugar donde se hace exactamente lo que viste en la pantalla de Firebase: `import initializeApp ... const app = initializeApp(firebaseConfig)`. Aquí también se crean `auth` (para el login) y `db` (para Firestore), y se exportan para que los use el resto del sitio.
- Cada página HTML (**index.html**, **registro.html**, **dashboard.html**) trae, al final del <body>, su propia etiqueta `<script type="module" src="js/login.js"></script>` (o registro.js / dashboard.js según la página). Esa es la etiqueta <script type="module"> equivalente a la que viste en Firebase, solo que en vez de tener todo el código adentro, apunta a un archivo .js aparte con `src`.
- Ese archivo .js de cada página (ej. `login.js`) hace `import { auth, db } from "./firebase-init.js"` para reutilizar la conexión a Firebase ya creada, en lugar de repetir initializeApp en cada página.
- El atributo `type="module"` es obligatorio: es lo que le permite a un archivo .js usar `import`. Si se quita, el sitio deja de funcionar.

En resumen: no tienes que escribir ni pegar ningún <script> nuevo. El único archivo que editaste es firebase-config.js (paso 5); toda la conexión con Firebase y la carga de módulos ya está resuelta en el código del proyecto.

7. Estructura del proyecto

Archivo / carpeta	Para qué sirve
index.html	Pantalla de inicio de sesión
registro.html	Formulario del trabajador
dashboard.html	Panel con filtros para el ejecutivo
css/styles.css	Estilos visuales de todo el sitio
js/firebase-config.js	Tus claves de Firebase (pasos 2 y 5)

js/firebase-init.js	Inicializa Firebase (no se edita)
js/auth-guard.js	Verifica sesión y rol en cada página
js/login.js, registro.js, dashboard.js	Lógica de cada pantalla
firestore.rules	Reglas de seguridad (paso 4)

8. Subir el proyecto con GitHub Desktop y activar GitHub Pages

El repositorio en GitHub.com ya está creado. Ahora, usando la app **GitHub Desktop** (sin usar la terminal):

1. Abre **GitHub Desktop** e inicia sesión con tu cuenta de GitHub si no lo has hecho.
2. Clic en **File** → **Clone repository** (o el botón "Clone a repository from the Internet..." en la pantalla de inicio).
3. En la pestaña "GitHub.com", busca y selecciona el repositorio que ya creaste (ej. "registro-piezas"). En "Local path" elige dónde quieres guardarlo en tu computadora y clic en **Clone**.
4. GitHub Desktop crea una carpeta vacía (con solo un README, si lo generaste al crear el repo) en esa ruta. Copia **todo el contenido** de la carpeta del proyecto (index.html, registro.html, dashboard.html, css/, js/, firestore.rules, el PDF, etc.) dentro de esa carpeta clonada.
5. Regresa a GitHub Desktop: en automático va a detectar y listar todos los archivos nuevos en la pestaña **Changes**.
6. Abajo a la izquierda, en el cuadro de resumen, escribe un mensaje como **"Primera versión"** y clic en el botón azul **"Commit to main"**.
7. Arriba, clic en **"Push origin"** para subir los cambios a GitHub.com.
8. En el navegador, entra al repositorio en **github.com** → **Settings** → **Pages**.
9. En "Source", elige la rama **main** y la carpeta **/ (root)**. Guarda.
10. Espera 1-2 minutos. GitHub te dará una URL como **https://TU_USUARIO.github.io/registro-piezas/**. Esa es la dirección del sitio.

> Cada vez que edites algún archivo después de esto (por ejemplo, otro ajuste a la guía o al código), el flujo en GitHub Desktop es siempre el mismo: guardar el archivo → verlo aparecer en "Changes" → escribir un mensaje → "Commit to main" → "Push origin". El sitio en GitHub Pages se actualiza solo unos minutos después de cada Push.

9. Probar que todo funcione

1. Abre la URL del sitio y entra con el correo/contraseña de un usuario con rol "trabajador".
2. Debe llevarte directo al formulario. Llena los datos de una pieza de prueba y da clic en Registrar.
3. En Firebase, ve a Firestore → colección "registros" y confirma que apareció el documento nuevo.
4. Cierra sesión y entra con un usuario con rol "ejecutivo". Debe llevarte al dashboard y mostrar ese registro en la tabla.
5. Prueba los filtros (folio, zona, trabajador, rango de fechas) para confirmar que funcionan.

10. Cómo agregar un usuario nuevo más adelante

1. Authentication → Users → Agregar usuario (correo y contraseña).
2. Copiar su UID.
3. Firestore → colección "usuarios" → crear documento con ese UID como ID, y los campos nombre y rol.

No se necesita tocar código ni volver a publicar el sitio para agregar usuarios: todo pasa dentro de la consola de Firebase.

11. Siguientes pasos posibles (opcional, más adelante)

- Agregar una pantalla dentro del dashboard para dar de alta usuarios sin entrar a la consola de Firebase.
- Agregar gráficas visuales (ej. piezas por zona, por día) además de la tabla filtrable.
- Agregar más zonas o renombrarlas si cambian las áreas físicas.
- Exportar los registros filtrados a Excel/CSV desde el dashboard.